基于 PYNQ Z2 的通用串口监测与异常取证系统

技术报告

摘要

本项目面向机器人、多轴运动与传感器融合设备的调试需求，研究基于 FPGA 的可扩展信号采集与故障记录方法，以平衡车作为验证载体。间歇性异常的分析需要关联触发前后的状态、通信质量与恢复过程。系统基于 PYNQ-Z2，将原始字节、逐字节硬件时间、协议字段、规则事件和异常前后窗口关联保存，使一次告警能够回到原始输入中复核。

系统采用 PL 与 PS 分工。PL 负责 UART 接收、100 MHz 相对计数、FIFO 及可配置二进制解析，利用 8 条独立规则通道并行判断字段异常；PS 负责文本等通用协议解析、连续多事件取证、持久保存、独立回放和 Notebook 可视化。同一份 URX2 固件可在停止采集后切换硬件二进制规则与软件文本规则，无需为每种设备重新生成位流。

真实 F103 平衡小车完成 300.045 秒联合采集，保存 15028 帧、721344 字节，校验、缺号和采集溢出均为 0。EGo1 文本测试源在 9600 与 115200 两档各接收 600 帧，与电脑独立接收及参考流逐字节一致。二进制测试完成 800 帧，两档八路场景各保存 16 个完整设备事件窗口，持续越限正确检出、短毛刺有效过滤。受控过载与文件系统错误测试验证了本地丢失识别和保存重试。

100 MHz 实现占用 15971 LUT、13887 FF 和 22.5 个 BRAM36 等效资源，DSP 为 0。系统完成正常关机后断电、人工恢复和重新采集，1228 个文件摘要一致；接收、检测、保存、回放和恢复形成完整工作流程。配套九项 Skill 提供选题规划、优化取舍、体验设计及实施核验方法。

关键词 协议配置复用 FPGA 并行规则 硬件时间戳 连续异常取证 PYNQ

表 1 系统定位与验证范围

项目	范围
主平台	PYNQ-Z2 Zynq-7000 XC7Z020
输入与执行	单路 3.3 V UART 8N1；只读监听
设备与测试源	F103 平衡小车；EGo1 二进制及 LAB 文本测试源

项目	范围
核心能力	可配置解析、8 路并行规则、连续事件取证与回放

目录

摘要 1

1 应用背景与设计目标 4

2 总体架构与软硬件分工 6

3 字节采集与可配置接口 8

4 八路规则并行检测设计 9

5 连续多事件取证与状态评价 10

6 真实小车与合成测试源接入 11

7 数据通路与界面优化 12

8 实验方法与一致性校验 13

9 真实小车运行与实时界面结果 14

10 并行与连续取证验证 15

11 协议复用与失败恢复结果 17

12 资源开销与性能分析 19

13 大模型协作记录与技能提炼 21

14 从零开始的复现步骤 25

15 总结与展望 29

参考资料 30

1 应用背景与设计目标

面向机器人与精密设备的故障记录需求

机器人关节调试、多轴运动平台和惯性传感器实验需要同时观察位置、速度、电流、姿态及通信状态。短时速度波动、测量跳变或数据迟到可能影响控制效果；排查时不仅要找到越限值，还需要知道各信号出现的先后顺序，以及异常发生前后的控制状态。ABB 的机器人信号分析资料强调信号顺序和时间戳的重要性，Beckhoff 也将轴位置与运动状态记录纳入调试工具。[7、9]

随着接入的传感器和运动轴增加，故障记录逐渐成为多源数据的时间关联问题。例如，同一次姿态突变可能伴随电流变化、编码器反馈异常或通信中断。将这些信号放在共同时间线上，保留触发前后的连续记录，有助于工程师建立和检验原因假设。ROS 2 追踪研究同样表明，运行时执行信息和时间关系能够帮助分析复杂机器人系统的性能问题。[10]

目标用户与验证载体

项目面向高校实验室、机器人原型研发团队，以及缺少完整专用调试工具链的中小制造与集成团队。典型任务是接入自研控制器、不同来源的传感器和运动模块，重复试验后定位间歇性问题。设计重点为协议可配置、模块可扩展、记录可复核，使团队能够围绕自己的设备建立调试流程。

平衡车用于验证这一路线的基础闭环。其姿态、编码器、PWM 与控制标志能够在可观察的实体系统中形成关联，便于检查接入、规则判断、重复故障记录与结果回放。当前实现以单通道 UART 和八路规则完成平台验证，后续向机器人所需的更高输入负载与多源同步扩展。

FPGA 并发的设计依据

当多路数据持续到达且时间一致性要求提高时，采集、校验和规则判断需要同时推进。FPGA 可为不同输入设置独立接收通路，并以共同计数器标记接收时间；多条规则拥有独立状态，可在同一帧上同时判断。PS 负责配置、文件管理与展示，形成硬件确定性处理和软件适配的分工。NI 的 FPGA 设计资料也将独立并行循环与流水线用于采集和处理通路优化。[11]

现有方法与项目定位

机器人与工业测量领域已有成熟的信号记录、追踪和采集工具。ABB RobotStudio 可记录和显示运动及 I/O 信号；Universal Robots 提供日志、飞行记录和实时数据分析；NI 提供基于可重配置硬件的波形采集范例。本项目据此将目标落实为面向异构自研设备的可配置外部记录平台，优先打通设备字段、规则事件与前后窗口之间的关联。[7、8、12]

表 2 相关工具与本项目的技术侧重

方向或工具	已有能力	本项目的技术侧重
ABB RobotStudio 与 UR Log Viewer	面向相应机器人平台的信号、日志及运行数据分析。[7、8]	通过协议配置接入自研控制器及不同来源的数字遥测。
Beckhoff TwinCAT Scope	记录轴变量并显示位置、速度等运动过程。[9]	将字段规则、异常窗口和原始输入关联为可复核记录。
ROS 2 追踪	采集运行时执行信息，分析消息与处理延迟。[10]	从外部数字接口记录数据，与软件侧分析配合定位问题。
NI 可重配置采集	FPGA 采集、连续数据传输与波形记录。[11、12]	以 PYNQ 与配置化协议实现便于修改和复用的验证平台。
串口与逻辑分析	观察字节、设备日志、电气时序与总线波形。	围绕较长业务过程保留重复事件及前后文。
本系统	只读采集、硬件时间、八路规则、连续取证与回放。	为实验室与中小研发团队提供可扩展的故障记录流程。

设计目标与主要创新点

项目的总体目标是建立面向高性能机器人等精密设备的 FPGA 信号采集与故障记录平台。当前成果围绕三项技术展开：同一固件通过配置接入定长二进制与文本协议；独立硬件规则保留同帧全部命中；原始字节、硬件时间及前后窗口关联保存，并支持失败请求重试。

接入采用只读监听，JSON 描述分帧、字段、校验和规则，固定整数协议由 FPGA 处理，文本及复杂格式由 PS 处理。连续事件独立归档，采集质量与设备状态分别评价。平衡车和合成测试源共同验证协议复用、并发检测与取证流程，为多源设备接入提供基础。[3—6]

2 总体架构与软硬件分工

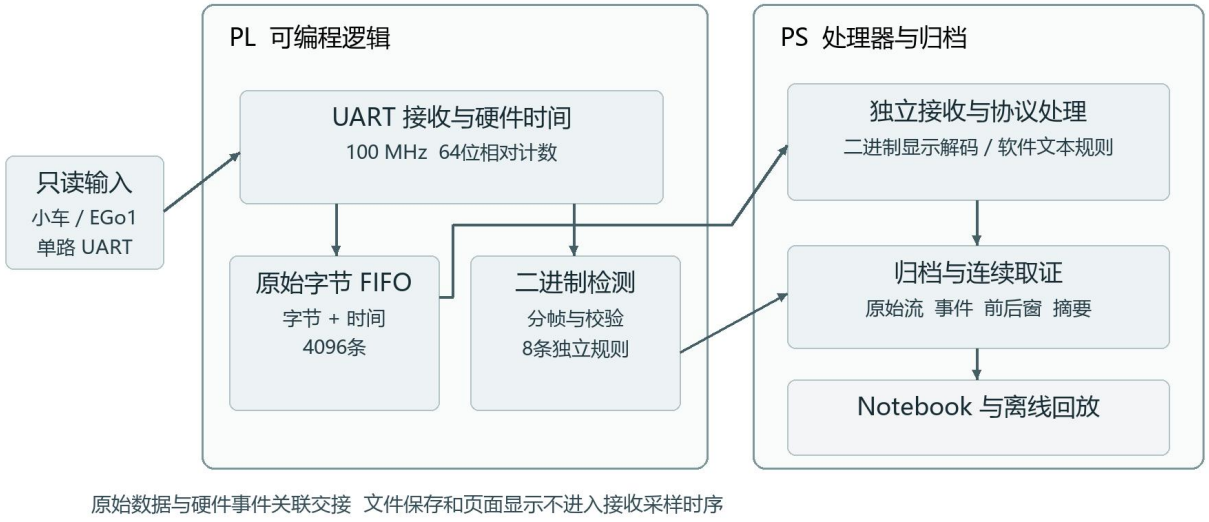


图 1 系统数据流与 PL PS 边界 文本解析和文件保存位于 PS

输入进入 UART 接收器后分成原始数据与检测两条路径。原始路径保存字节和 64 位相对时间；检测路径在硬件模式下完成定长分帧、校验、字段规范化及规则判断。硬件事件以批次进入事件 FIFO；PS 在相应原始字节已经读出后，再将事件交给取证核心，确保事件所指数据可以被定位。[3]

表 3 关键模块的职责

模块	位置	职责
UART 与时间	PL	单时钟域接收；逐字节记录相对计数；上报停止位和溢出
二进制检测	PL	分帧、校验、整数提取及 8 条规则各自的持续状态
接收泵与协议	PS	有界读取、队列转交、解码及软件模式规则
归档与窗口	PS	磁盘索引、前后窗导出、失败请求和摘要回读
Notebook	浏览器与 PS	实时状态、历史浏览和导出；不驱动硬件采样

两种规则执行方式共享同一套原始数据格式、事件格式和窗口保存机制。硬件模式选择可编译的二进制配置；软件通用模式旁路硬件解析器，保留 UART、硬件时间及 FIFO，由 PS 解析文本、浮点或显式注册的专用协议。切换模式前必须停止并排空接收，重新检查配置。

软硬件分工兼顾处理确定性与协议适配。FPGA 将字段判断分配给 8 个具有独立状态的规则实例，实现同帧多规则并行判断和事件交接；PS 承担通用解析、文件管理及展示，协议切换复用原始记录与事件接口。[1、2]

3 字节采集与可配置接口

硬件时间与原始数据路径

接收接口为单通道 3.3 V UART、8N1，支持 9600、19200、38400、57600 和 115200 波特率。接收和接口逻辑工作在 100 MHz 时钟域，64 位计数随 PL 运行递增，每个进入 FIFO 的字节附带一个时间值，用于定位帧、事件及窗口。原始 FIFO 深度 4096 条字节记录。

AXI-Lite 基址为 0x43C10000，映射范围 64 KiB。URX2 的 ID 为 0x32585255，ABI 为 2.0。驱动核对身份、能力、时钟、容量及位流配对关系，再允许访问。读取字节后使用稳定时间快照组成 64 位时间，避免高低 32 位跨界时读到不一致值。停止与清除操作分离：停止保留待读数据，清除只在满足空闲约束时执行。[3]

协议配置与执行方式

表 4 URX2 硬件模式的配置范围

配置项	支持范围
帧与同步	定长 1 至 512 字节；包头 1 至 4 字节；最多 4 段固定常量，每段最多 4 字节
字段	最多 16 个 8 位、16 位或 32 位有符号及无符号整数，支持大小端
校验	无校验、XOR8、SUM8、CRC16 MODBUS、CRC32 ISO HDLC
规则	最多 8 条独立范围、比较、位掩码及有效性规则
序号与设备时间	最多 32 位；缺少相关字段时保留能力不足状态
软件模式	文本、float32、64 位字段、复杂插件及行为扩展

配置经编译、写入、回读和 COMMIT 后启用。驱动在接收停止、FIFO 排空且解析空闲时切换配置和波特率，并离线检查字段、校验范围与规则引用。专用解析插件通过显式注册接入。

软件分别保存原始值、换算值与有效性。数值规则使用换算后的单位，位规则使用原始整数，无效字段记录为 null。硬件编译器将支持的正比例缩放和偏移转换为整数阈值，FPGA 直接比较原始整数。

4 八路规则并行检测设计



图 2 同一帧进入 8 条独立规则通道 多条命中以同一事件批保留

分帧器和校验器按字节处理输入，字段规范化后送往 8 个独立规则实例。每条通道保存比较条件、持续确认起点和告警状态；同帧多规则命中通过事件位图一起保留。9600 和 115200 两档实板测试均取得两批八规则同帧触发、16 个完整设备窗口，电脑独立回放一致。[3]

持续确认与恢复再触发

比较条件首次成立时记录起始时间，后续有效样本持续满足条件并达到配置时长后触发。条件恢复时解除该规则的持续状态与锁存，下一次异常可形成新事件。规则连续性按字段有效性和输入时间管理，持续告警在恢复前保持同一事件状态。

持续确认按硬件时间和有效样本到达时刻计算。示例配置为 100 ms 持续阈值，10 Hz 输入的观察粒度为 100 ms，确认结果落在满足持续条件的后续帧。事件使用输入字节的原始时间，与界面刷新时刻分别记录。

事件批与背压

硬件事件携带规则位图、原始字段值、触发时间和字节范围。事件 FIFO 容量 64 批，每批可包含 8 条规则。驱动分别读取原始、事件及解析状态，将通路溢出归入采集质量，并继续按有效业务数据评价设备状态。

5 连续多事件取证与状态评价

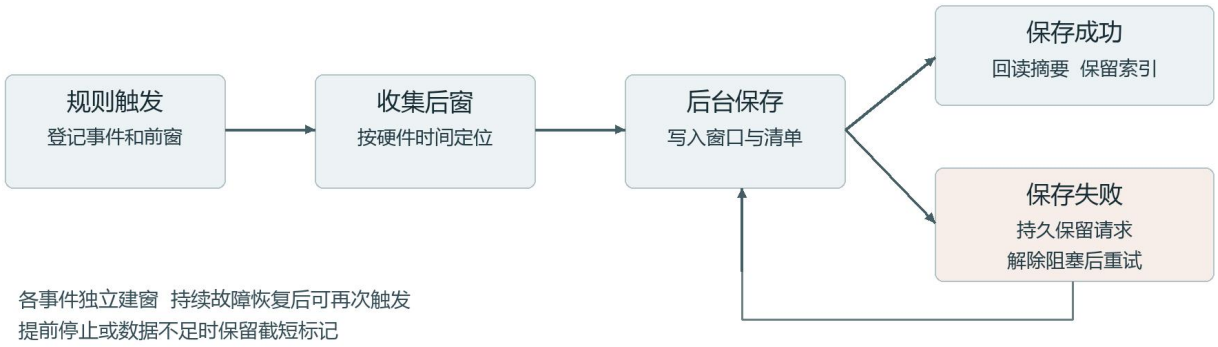


图 3 独立窗口的保存与重试流程 原始流和事件日志持续保留

新会话采用连续取证，每个事件定位默认前 5 秒、后 5 秒的原始数据。窗口支持重叠，同一时刻的不同规则独立留证；同一规则恢复后再次异常自动建立新记录。Notebook 待处理容量 32，总窗口上限 100，达到上限时记录未建窗数量并提示。[5]

原始流顺序追加到磁盘，并维护帧时间及文件位置索引。导出按时间定位数据段，每个窗口保存范围、完成状态、尝试编号和文件摘要。起始时间、停止时间及采集状态共同确定窗口实际边界。

表 5 不同状态分别评价

维度	回答的问题	典型含义
采集完整性	监测端是否丢失或错误接收	FIFO 溢出时 FAIL
协议完整性	现有校验及序号是否符合预期	故意坏 CRC 可被正确识别为 FAIL
设备字段状态	有效业务数据是否触发规则	有明确字段告警为 FAIL；丢失且无证据为 UNKNOWN
证据保存	应保存的窗口是否成功持久化	SAVE_FAILED 不等于原始流已经丢失

后台任务负责保存，失败请求持久保留，支持运行中或停止后的重试。保存成功更新窗口台账和报告，设备事件保持原有身份。界面分别展示设备字段告警、通信提示与保存状态，便于定位当前处理阶段。

6 真实小车与合成测试源接入

F103 平衡小车

真实业务输入为 STM32F103C8T6 小车的 CAR-TLM1 遥测。USART1_TX 使用 PA9，与 PYNQ 接收输入及公共地连接。协议为 115200、8N1、48 字节定长帧，默认约 50 Hz。小车沿用原实体按钮、姿态保护、看门狗和电机控制参数。[4]

表 6 CAR TLM1 帧结构

偏移	内容	说明
0 至 3	同步与格式	A5 5A、版本 01、长度 0x30
4 至 15	序号和时间	sequence、uptime_ms、imu_sample_count，各 32 位
16 至 31	姿态与编码器	倾角、角速度、E1/E2 累计计数，有符号 32 位
32 至 43	控制与状态	两路 PWM、flags、fault_code、tx_skipped、imu_age_ms，各 16 位
44 至 47	CRC32	对前 44 字节计算 CRC32 ISO HDLC，小端存放

负数按补码、小端逐字节打包。flags 提供字段有效性、控制活动及驱动使能状态。倾角原始值以毫度保存，显示时换算为度；PWM 使用定时器比较计数，编码器保存累计计数。

可配置测试源

EGo1 提供可重复的二进制和 LAB 文本测试数据。二进制格式兼容 CAR-TLM1，提供正常、校验、序号、断流、持续越限、短毛刺及八规则场景；LAB 文本提供序号、时间、温度、电压和状态字段，用于协议切换与字段适配测试。

EGo1 J5 物理 15 脚 B11 连接 PYNQ PMODA 物理 1 脚 Y18，J5 物理 33 或 34 脚公共地连接 PMODA 物理 5 或 11 脚。小车 PA9 与 EGo1 TX 轮流接入同一接收脚，两板独立供电。接收结果通过参考数据和电脑回放核验。

7 数据通路与界面优化

连续输入的接收同步

UART 接收器持续跟踪线路状态，采集使能控制完整字节入队；会话清除与字节解码状态分离。更换波特率时执行接收器复位及有界同步等待，使连续遥测中的会话启停保持字节同步。[3]

表 7 关键优化与验证结果

优化环节	实现方式	验证结果
连续流同步	后台字节跟踪，解码与采集门控分离	160 组相位、320 次启停、37 轮实车会话
接收与存储	独立接收线程、1024 项有界队列	180 秒静态采集无 FIFO 溢出
队列交接	最多 256 字节或 50 ms 片段合并	测试队列峰值 61/1024
页面传输	完整快照无损压缩与 Base64 传输	2000 帧逐字段解码一致
状态显示	检测引擎状态与事件联动	60 秒复测页面状态正常

独立接收线程保持原始字节顺序和逐字节时间，片段合并减少队列交接开销。队列容量 1024 项，单片段最多 256 字节或 50 ms，队列状态由运行监测持续跟踪。

实时显示与数据保存

接收、归档和页面刷新独立运行。Notebook 以完整快照更新界面，使用无损压缩降低传输量；协议选择同步切换字段名、单位与默认曲线。显示状态取自检测引擎及实际事件，使采集、通信和设备告警保持一致的语义。

8 实验方法与一致性校验

实验采用发送参考、PYNQ 原始记录和电脑独立回放进行对照。PYNQ 保存 raw.bin、逐字节 timestamps.bin、frames.jsonl、events.jsonl、硬件事件批及窗口清单。电脑从相同字节和时间重新解析、执行规则，逐项比较帧、事件及文件摘要。

表 8 分层验证方法

验证层级	方法	校验内容
RTL 仿真	串口、AXI、自检与相位扫描	接口握手、边界及规则输出
实板采集	设备输入与可重复测试流	字节、计数、触发与保存结果
独立回放	原始字节与硬件时间重新解码	字段、事件及窗口摘要一致性
浏览器交互	协议切换、实时状态与窗口查看	字段映射、状态与快照解码

测试配置与输入

测试采用同一份 URX2 固件。二进制四轮测试覆盖 9600 与 115200 两档八路、持续越限和短毛刺，共 800 帧；实际 Notebook 用于选择配置、观察状态和查看窗口。文本测试通过软件通用模式接收，两档各 600 帧。

小车 300 秒联合采集在 115200、约 50 Hz 下进行，遥测与浏览器状态同步记录。硬件时间以本次 PL 运行为参考；发送区间、首帧等待和结束阶段按事件位置标识。

异常与完整性判定

规则事件与采集质量分别判定：注入越限应产生对应字段告警，短毛刺应被持续确认过滤；过载应产生采集溢出状态。无校验或无序号时，对应检查显示 UNKNOWN 或不具备判断条件。

9 真实小车运行与实时界面结果

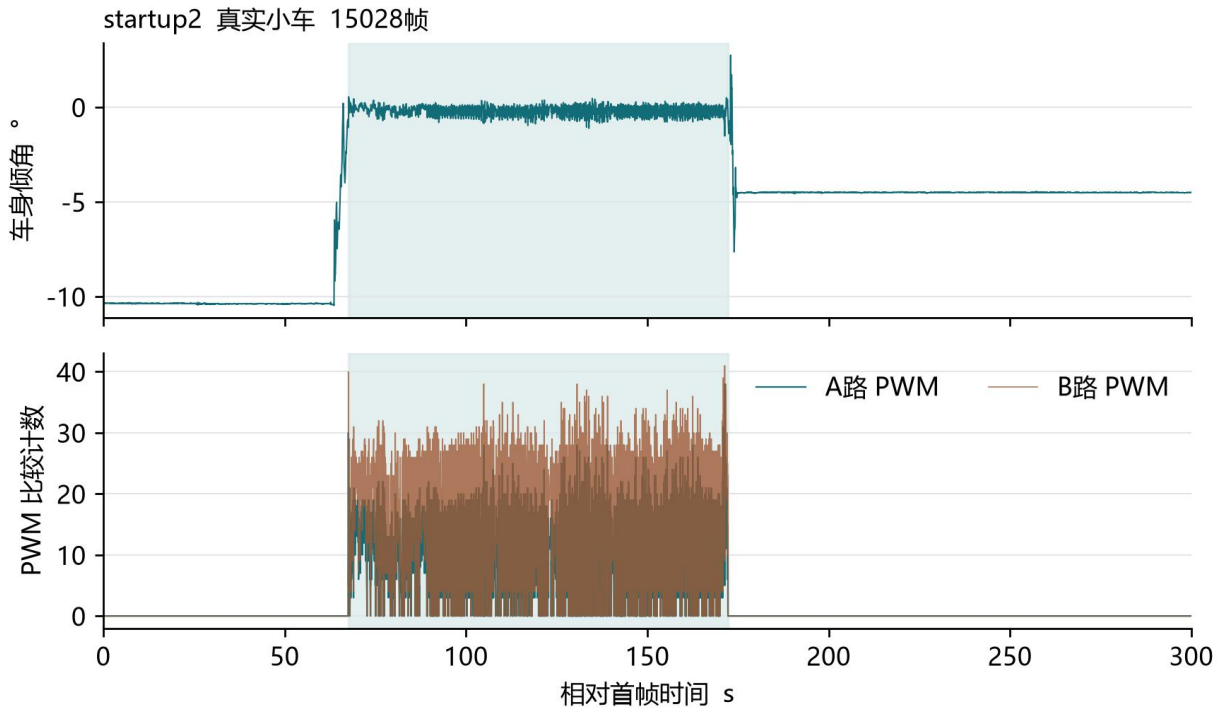


图 4 真实小车姿态与 PWM 记录 阴影为控制活动观察区间 输入 115200 约 50 Hz

联合采集持续 300.045 秒，共 15028 帧、721344 字节。采集完整性和协议完整性均通过，CRC、缺号、重复、停止位及各级硬件溢出均为 0，电脑独立回放一致。控制活动标志在 5245 帧中连续置位，对应首帧后 67.535 至 172.207 秒，观察跨度 104.672 秒。控制期间倾角范围为 −1.517°至 0.548°。

运行期间驱动按控制策略启停，其中 352 帧驱动关闭且 PWM 为 0。控制活动结束后继续记录 6400 帧、约 127.77 秒，两路 PWM 及运行、驱动标志均为 0。

实时 Notebook 与采集同步运行，浏览器取得 267 个观察样本，202 次显示新帧。界面状态与检测事件联动；另一次 60.050 秒静态检查取得 3006 帧，54 个浏览器样本显示正常，通信与采集错误计数为 0。

10 并行与连续取证验证

EGo1 二进制场景在约第 6 秒和第 14 秒将 fault_code 置为 20，使 parallel_0 至 parallel_7 八条规则同时成立。每轮共 16 条设备事件、16 个完整前后 5 秒窗口，电脑独立回放一致。两批触发之间条件恢复，系统自动继续记录。

二进制四轮测试每轮 200 帧、9600 字节，共 800 帧、38400 字节，与发送参考逐字节一致。两档八路各保存 16 个完整窗口，持续越限产生 1 条设备事件，短毛刺过滤有效；停止位、校验、序号及各级溢出错误均为 0。

二进制检测结果

表 9 二进制测试结果 每轮 200 帧

场景	设备事件与窗口	独立核验
9600 八路	两批各 8 条；16 完整窗口	原始参考一致；事件回放一致
115200 八路	两批各 8 条；16 完整窗口	原始参考一致；事件回放一致
持续越限	1 条设备事件；完整窗口	持续确认正确检出
短毛刺	0 条设备越限事件	未误报；原始回放一致

设备字段事件与通信提示分类记录。每次触发关联规则、硬件时间和独立窗口，支持同帧多规则以及恢复后的再次触发。

连续记录的实际操作界面



图 5 八路取证记录的回放界面 展示协议、状态、触发原因、时间及窗口保存结果

界面按设备协议切换字段，选择事件即可定位前后 5 秒。同批命中显示数量，条件恢复后自动记录下一次异常。曲线可选平滑显示，取值、检测和导出使用原始数据；采集控制与小车原启停操作分开。

11 协议复用与失败恢复结果

同一位流的两种协议

同一 URX2 固件以软件模式接收 LAB 文本（@LAB,seq,ms,temp_mdeg,voltage_mv,status*HH，XOR8）。9600 与 115200 两档各 600 帧、18978 字节，校验、序号、停止位及溢出错误为 0，原始数据逐字节一致。[4]

20 帧离线样例检查了无校验、无序号配置：对应能力标志为 false，报告及页面显示不具备判断条件。

采集过载识别

正常 LAB 输入受控停读 16 秒，FIFO 达到 4096 条。硬件计数 6178 字节，保留 5344、丢失 834；解析 172 帧，校验错误 1、缺号 28。系统标记采集 FAIL、协议 FAIL、设备 UNKNOWN，完成分层归因。[5]

保存失败与持久重试

表 10 同一保存失败记录的状态变化

阶段	原始数据	窗口状态	解释
首次保存	200 帧完整	2 窗口失败	路径阻塞，返回 ENOTDIR
第二次尝试	摘要不变	仍失败	保留请求，继续重试
第三次尝试	摘要不变	2 窗口成功	解除阻塞，回读通过
设备状态	事件保留	告警保留	保存结果独立更新

以 ENOTDIR 目录错误验证持久重试。解除阻塞后两个窗口回读通过，200 帧原始数据摘要不变；窗口按实际时间范围标记。

正常关机后的恢复

正常关机后完成断电、重新上电。人工恢复网络并加载 URX2，1228 个程序、配置及结果文件摘要一致，保存记录正常打开。

恢复后取得 200 帧、9600 字节，两批各 8 条规则、16 个完整窗口回读及回放通过，接收、校验、序号与溢出错误为 0。

12 资源开销与性能分析

表 11 100 MHz 布局布线结果

指标	结果	说明
LUT	15971 30.02%	基于 XC7Z020 可用资源
触发器	13887 13.05%	包含解析和规则等逻辑
BRAM36 等效	22.5 16.07%	缓存和事件通路
DSP	0	本实现不以 DSP 作规则比较
建立时间 WNS	0.000 ns	TNS 为 0，失败端点为 0
保持时间 WHS	0.044 ns	THS 为 0，失败端点为 0
约束与检查	DRC 错误 0	无未约束内部端点；保留 XDCB-5 查询告警

100 MHz 布局布线报告中，建立与保持检查均无失败端点，DRC 错误为 0。UART 异步输入采用两级同步，CDC 报告保留相应同步结构信息。

输入负载与缓存配置

48 字节帧以 50 Hz 发送，业务数据率为 2400 B/s。8N1 每字节占 10 位，线路负载约 24000 bit/s，占 115200 波特率的 20.83%。4096 条原始 FIFO 与独立 PS 接收线程共同吸收处理时间波动，原始、事件和解析通路分别统计溢出。

页面传输优化

在 2000 帧快照测试中，原 JSON 约 2510942 字节，zlib 压缩及 Base64 编码后传输约 192718 个 ASCII 字符，体积约减少 92.3%。浏览器解码后逐字段一致，损坏数据校验有效。页面目标刷新间隔为 0.5 秒，后台接收独立运行。[6]

优化围绕数据通路分工展开：FPGA 以独立规则状态提供同帧并行判断，PS 以配置化解析支持格式适配，索引导出和快照压缩降低文件定位与页面传输开销。资源、输入负载与页面传输量共同给出实现代价和运行条件。

优化前后的性能与资源对比

资源对比采用相同 XC7Z020 器件、Vivado 2025.2 及 100 MHz 目标时钟的布局布线报告。优化前为字节采集与 PS 规则方案，优化后在 PL 集成可配置解析与八条独立规则；资源增量用于检测通路、配置及事件缓存。

表 12 检测下沉前后的实现代价

指标	字节采集与 PS 规则	PL 集成八路规则
LUT	803（1.51%）	15971（30.02%）
触发器	1017（0.96%）	13887（13.05%）
BRAM36 等效	8（5.71%）	22.5（16.07%）
DSP	0	0
目标时钟	100 MHz	100 MHz
规则执行位置	PS 按规则计算	8 条独立硬件通道
同帧事件交接	软件生成事件	一次保留 8 位命中及共同时间

表 13 页面传输与持续流同步优化对比

比较项目与条件	优化前	优化后
相同 2000 帧完整快照	JSON 2510942 B	压缩加 Base64 192718 B
快照传输量	100%	7.68%，减少约 92.3%
快照内容	完整字段	无损解码后逐字段一致
连续流字节内部开启采集	解码复位耦合，出现停止位错误	解码持续跟踪，完整字节入队
同步验证	半位偏移激励复现问题	160 组相位、320 次启停通过

优化效果分别体现在硬件规则并行、快照传输量和连续流启停正确性。115200、约 50 Hz 小车负载下，最终系统完成 300.045 秒、15028 帧采集，校验、序号和溢出错误均为 0。

13 大模型协作记录与技能提炼

协作采用任务分解、工具调用与结果复核的工作流。设计者提出目标与操作约束，模型阅读契约、生成实现与验证程序，通过工具取得运行结果，再依据结果修正。设备接线、实体按钮和实物观察由设计者完成；模型负责代码与结果核对。

需求提示与模型回答

以下提示词摘录自实际协作；模型回答以已经实施的方案摘要呈现。记录保留需求、处理与验证之间的对应关系。

案例一 通用接入与软硬件分工

提示词：“进行 FPGA 集成，按照你的推荐来，先完成离板验证”。需求同时要求通过配置接入不同协议，保持现有可视化接口。

模型回答摘要：将定长分帧、校验、整数提取和八路规则放入 PL；PS 保留文本、浮点及插件解析，统一 ByteChunk、DecodedFrame、DetectionEvent 与归档接口。先以独立参考生成输入，再检查 RTL、AXI 接口和实现资源，最后进入实板验证。

实施结果：形成 URX2 硬件与配置编译器，同一位流完成 CAR-TLM1 二进制与 LAB 文本接收；9600 和 115200 八路测试各产生两批同时命中，共 16 个完整设备窗口。

案例二 连续故障记录与界面可读性

提示词：“窗口多没问题，我们就要记录不同时间的故障，不能只记录一次就必须人工恢复，但记录时清楚的加上记录故障和触发时间，提高可读性，不能让用户去适应我们带来的繁琐”。

模型回答摘要：按规则维护恢复和再触发状态，每次异常独立编号；列表显示故障名称、触发时间、触发依据及保存状态。后台采集、窗口保存与页面刷新分离，持续同一告警保持一个事件，恢复后再次出现自动记录。

实施结果：两批八规则事件连续归档，真实小车两次倾角告警自动形成独立前后窗；保存路径错误解除后，持久请求可继续重试并通过摘要回读。

自我纠错与工具工作流

连续 UART 输入的边界实验推动了接收逻辑修正。初始实现将解码器复位与采集开关耦合，验证发现字节内部启动可引入停止位错误。模型依据复现波形修改解码与记录的边界，并扩大验证相位和真实启停覆盖。

表 14 接收同步的纠错轨迹

环节	输入或观察	处理与输出
形成初始实现	采集使能同时控制 UART 解码复位	建立可运行接收与会话逻辑
工具反馈	正常 0x00 流，半位偏移开启时出现停止位错误	把问题定位到采集端的字节同步
模型修正	字节解码应保持线路同步	解码常运行，使能只门控完整字节入队
扩大反证	相位、启停、换波特率及真实坏停止位	160 组相位、320 次启停和 3302 项 AXI 检查通过
真实输入复核	F103 持续遥测 37 轮启停	2312 有效帧、8 次字节中途启动见证，回放一致

另一项纠错发生在显示状态：页面原先依据帧时间差推断断流，与检测引擎的事件状态不一致。修正后，状态显示直接采用引擎活动状态及实际事件；60.050 秒、3006 帧复测中，54 个浏览器观察样本显示正常。

智能体工作流

第一步固定输入输出、配置与判据；第二步阅读对应 RTL、驱动和记录；第三步生成实现及独立参考；第四步通过 Python 检查、XSim、Vivado 实现报告取得工具反馈；第五步在实板上记录原始流与硬件事件；第六步由电脑独立回放核对字段、事件和摘要。任何关键改动均回到对应检查环节，最后整理可复用步骤。

该流程将模型回答转成可检查的工程产物：代码说明实现方式，工具输出回答是否满足判据，原始字节和时间索引支持再次核对。

技能包的提炼过程

九项 Skill 按统一结构交付，覆盖工程决策和实施核验。提炼顺序为观察任务障碍、抽取可重复步骤、定义输入输出与失败条件、运行正负向检查，再把器件和协议参数作为适配项列出。

表 15 观察来源与技能形成

失败或观察	提炼的流程与对应 Skill	验证与适用范围
临近截止需确定优先级	fpga-project-plan：目标、资源、工期与最小闭环	按赛题交付与现有证据走查；任务和评分目标为输入
硬件下沉与页面延迟是不同问题	fpga-optimization-plan：定位瓶颈后选方案	资源报告及快照对照；按新输入负载重新选指标
相似窗口不易辨认	monitoring-ux-design：故障、时间、依据、保存状态优先	实际 Notebook 及重复事件检查；字段随设备配置
固件切换和缓冲区有状态	pynq-overlay-verify、pynq-dma-verify：身份及空闲先检查	配对摘要、寄存器与数据检查；按接口适配状态机
结果需能独立重算和完整交接	fpga-reference-verify、fpga-release-verify	参考模型、严格清单与负向篡改检查；文件格式由契约给定
运行环境和连续流影响结果	pynq-runtime-debug、pynq-uart-verify	解释器兼容、相位扫描及真实记录回放；板型和时间基准显式配置

配套工具完成 15 项正负向自检和 8 份记录核验，检查文件集合、摘要、路径、时间索引、能力标志及分层状态；九项 Skill 的标准入口、内部链接与解压文件清单通过检查。

使用时先从 skill/README.md 选择任务，再阅读对应 SKILL.md 的输入与前置条件，执行步骤并检查输出。涉及寄存器、缓冲区或文件格式时替换项目适配参数；选题、优化和交互 Skill 用于形成决策及验收任务，核验 Skill 用于执行相应检查。

```
python skill/pynq-uart-verify/scripts/self_test.py
python skill/pynq-uart-verify/scripts/check_project.py --project-root . --output build/skill_check_01
```

AMDROSS 套件使用心得与改进建议

本项目对 AMDROSS 智能体套件的使用观察主要集中在领域技能组织与工具入口衔接。套件围绕 IP 配置、仿真、时序检查、日志分析及硬件调试提供专门流程，有助于把 FPGA 开发经验整理为明确的输入、操作步骤和检查输出。相比单次代码生成，这种组织方式更便于智能体根据工具反馈推进工程任务。

在实际协作中，项目目标、软硬件划分和使用体验仍需结合设备需求确定；具体实现与验收通过 RTL、工程脚本、Vivado 报告、板端原始记录及独立回放完成。由此形成的心得是：智能体套件应同时支持任务执行和工程状态维护，让每次工具调用都能接续已有工作，并留下可复核的结果。

表 16 结合工程实践提出的 AMDROSS 改进建议

改进方向	建议机制	工程中的应用价值
统一工程状态	记录板卡、工具版本、固件、接口参数、活动任务及下一步操作条件。	切换协议和波特率时自动核对收发配置，减少重复确认。
关联验证结果	将仿真、实现和实板结果绑定源码、配置及固件摘要；改动后提示所需重测。	清楚判断当前实现的完成阶段，支持持续开发与交接。
边界验证模板	提供字节中途启动、FIFO 满、AXI 背压、时间跨界和保存失败等可配置激励。	将持续流同步等真实问题转为可重复验证的工程案例。
人工操作协同	生成接线、开关、按键和预期现象的简短操作卡；接收准备完成后再提示发送。	协调人机上板步骤，依据实际计数判断测试进度。
优化实验设计	提出并行化、流水线或硬件下沉时，同步给出基线、测量方法和性能资源指标。	用可比较的实验结果支持方案选择与资源取舍。

其中，统一工程状态与结果追溯建议优先实施。它能够贯通需求、代码、固件和验证记录，同时改善调试效率、团队交接与复现过程。这也成为本项目提炼 Skill 的方法依据：除执行技巧外，还应保留决策所需的上下文和完成判据。

14 从零开始的复现步骤

环境准备与目录定位

准备 PYNQ-Z2 及已写入 PYNQ 2.1 的 microSD 卡、USB 电源和网线。电脑安装 Vivado 2025.2（包含 Zynq-7000 支持）、PowerShell 7、Python 3 和 OpenSSH 客户端。板端使用 Python 3.6、PYNQ 2.1、ipywidgets 7.2.1 及可访问硬件的 Notebook 内核。将源码解压至纯英文无空格路径，例如 C:/fpga/uart_evidence，并在工程根目录执行命令。

工程采用按功能组织的目录，对应指南推荐结构如下。源码路径和命令保持一致，中文用于文档正文。

表 17 工程目录与推荐结构的对应关系

指南中的内容	本工程入口
README 与复现说明	README.md 及本章
src 设计源码	rtl/、scripts/uart_monitor/及 scripts/
sim 验证脚本与结果	tests/、scripts/simulate_urx2*.tcl、build/uart_detector2/
build 构建脚本和报告	scripts/build_urx2.tcl、build/vivado/
board 上板与配置	notebooks/、profiles/、firmware/
data 输入与参考结果	验证输入、build/uart_detector2/vectors/及 results/
skill 与 report	skill/及 deliverables/中的技术报告

先运行软件与配置检查。Python 命令需对应所选 Python 3 解释器；下列 PowerShell 命令中的 vivado 可替换为本机 vivado.bat 完整路径。

```
python --version
pwsh -File scripts/run_urx2.ps1 -Stage software
python scripts/uart_monitor_cli.py validate --profile profiles/car_tlm1.json --detector fpga
python scripts/uart_monitor_cli.py validate --profile profiles/lab_ascii_v1.json
```

完成条件：软件回归成功，两个配置输出 VALID。无板时可完成本页检查及仿真，硬件采集从接线步骤开始。

仿真构建与部署

在已安装 Vivado 的命令环境执行以下命令。Label 选择新的英文名称；构建目录按该名称建立，便于定位输出。工程使用随源码提供的板级定义与验证输入。

```
pwsh -File scripts/run_urx2.ps1 -Stage sim -Label reproduce_01
pwsh -File scripts/run_urx2.ps1 -Stage build -Label reproduce_01
```

仿真检查解析、校验、八路事件、AXI 背压及 FIFO 边界，并对照独立参考；构建检查时序、DRC、BRAM 和 8 个实际规则实例。实现报告输出至 build/vivado/18_uart_detector2_reproduce_01/reports/，配对固件输出至 build/pynq_z2/uart_detector2/reproduce_01/firmware/。

部署前在电脑准备运行目录。使用随提交提供的配对固件时，将下面$fw 改为该固件目录；自行构建时使用上述新输出。

```
$fw = 'build/pynq_z2/uart_detector2/reproduce_01/firmware'
New-Item -ItemType Directory -Force firmware | Out-Null
Copy-Item "$fw/uart_detector2.bit" firmware/
Copy-Item "$fw/uart_detector2.hwh" firmware/
Get-FileHash firmware/uart_detector2.bit -Algorithm SHA256
Get-FileHash firmware/uart_detector2.hwh -Algorithm SHA256
```

给 PYNQ 独立供电，BOOT 置 SD。电脑有线网卡设为 192.168.2.1/24，板端默认地址 192.168.2.99；已有网络可使用实际可达地址。用 SSH 登录板卡，建立工作目录后复制脚本、协议、Notebook 及固件。

```
ssh xilinx@192.168.2.99 "mkdir -p /home/xilinx/jupyter_notebooks/uart_evidence"
scp -r scripts profiles notebooks firmware xilinx@192.168.2.99:/home/xilinx/jupyter_notebooks/uart_evidence/
```

登录使用系统卡实际凭据。浏览器访问板卡 Jupyter，打开 uart_evidence/notebooks/UART_Monitor.ipynb；若需独立启动服务，在板端运行：

```
cd /home/xilinx/jupyter_notebooks/uart_evidence
sudo /opt/python3.6/bin/python3.6 -m notebook --allow-root --ip=0.0.0.0 --port=9090 --no-browser
```

使用终端给出的 token 访问 http://192.168.2.99:9090。保持此终端运行，另开终端进行后续命令。

首次加载与硬件连接

首次开机先保持外部 TX 断开。在有硬件权限的 Notebook 内核新建单元格，执行下列显式加载。把 expected_bit 和 expected_hwh 替换为电脑 Get-FileHash 输出的小写 SHA256。该步骤核对系统默认 base.bit 与目标配对文件，加载后保持接收停止。

```
from pathlib import Path
import hashlib, sys
from pynq import PL, Clocks, Bitstream
root = Path('/home/xilinx/jupyter_notebooks/uart_evidence')
sys.path.insert(0, str(root / 'scripts'))
from pynq_hardware_lease import HardwareLease
from uart_monitor.urx2 import URX2Transport
from uart_monitor.profile import load_profile
sha = lambda p: hashlib.sha256(p.read_bytes()).hexdigest()
expected_bit = '替换为电脑核对的 bit 摘要'
expected_hwh = '替换为电脑核对的 hwh 摘要'
base = Path('/opt/python3.6/lib/python3.6/site-packages/pynq/overlays/base/base.bit')
bit = root / 'firmware/uart_detector2.bit'
assert PL.bitfile_name == str(base)
assert sha(base) == ('c15081be382e21c397684632e274558ce5'
                     '57c27df1e958daa83c1386205a54a3')
assert sha(bit) == expected_bit
assert sha(bit.with_suffix('.hwh')) == expected_hwh
with HardwareLease():
    Clocks.fclk0_mhz = 100
    Bitstream(str(bit)).download()
    rx = URX2Transport(bit, load_profile(root/'profiles/car_tlm1.json'))
    state = rx.registers()
    assert all(state[k] == 0 for k in
               ('control', 'level', 'event_level', 'hardware_active'))
    rx.close()
```

此初始化适用于本项目 PYNQ 2.1 基础镜像。已加载 URX2 时直接使用 Notebook；其他启动应用应先按其流程停止并释放硬件。初始化完成后运行 Notebook 原入口单元格，显式选择“FPGA 并行规则 / URX2”。

连接小车时将 PA9 接 PMODA 物理 1 脚、公共地接物理 5 或 11 脚；连接 EGo1 时用 J5 物理 15 脚 TX 与 33 或 34 脚地。两个发送端轮流接入，各自供电，仅连接信号与地。小车先保持电机停止，监测端只接收。

运行检查与结果复核

小车场景：协议选择 car_tlm1.json，115200，FPGA 规则，记录 180 秒；依次点击检查配置、开始监测。数据持续更新后，按小车原按钮流程操作。停止接收后查看事件列表与窗口，再导出记录。需要自动运行时在板端工程根目录执行：

```
sudo /opt/python3.6/bin/python3.6 scripts/uart_monitor_cli.py capture --profile profiles/car_tlm1.json --detector fpga --bit firmware/uart_detector2.bit --seconds 180 --output results/car_01 --require-clean
```

二进制八路场景：从小车协议生成演示规则配置，8 条规则分别判断 fault_code 大于 0 至 7。将该配置复制至板端 profiles/并刷新 Notebook 入口。

```
python -c "import json; from pathlib import Path;
p=json.loads(Path('profiles/car_tlm1.json').read_text(encoding='utf-8')); p['id']='parallel8_demo';
p['rules']=[dict(id='parallel_'+str(i),kind='compare',field='fault_code',op='gt',value=i,duration_ms=0) for i in range(8)]; Path('profiles/parallel8_demo.json').write_text(json.dumps(p),encoding='utf-8')"
```

EGo1 通过 USB 连接电脑，在 Vivado 中选择 Open Hardware Manager、Open Target、Auto Connect，选中对应器件后以 Program Device 加载配套二进制发送器 bit。独立 SW0=1、SW1=0、SW2 至 SW4 均为 1，按 S0 一次运行 20 秒。PYNQ 先开始接收，配置 115200 及 parallel8_demo.json，覆盖发送前等待与结束后的后窗。预期 200 帧，两次各 8 规则同时触发，16 个完整设备窗口。

协议复用场景：停止并排空接收，把发送源换为 LAB 文本程序，选择 lab_ascii_v1.json 和“软件规则 / URX2 通用”。先 9600，再 115200；两次均设 SW1=1、SW2 至 SW4 为 0，SW0 分别为 0、1，按 S0 各发送 60 秒。预期每次 600 帧，字段切换为温度、电压与状态，固件保持同一份。

结果复核：Notebook 导出会话，或用 scp 将 results/car_01 复制到电脑 results/。以下命令检查原始摘要、重新解析和事件一致性，并生成可交互回放页；输出目录和 HTML 文件选择新名称。

```
python scripts/uart_monitor_cli.py verify --output results/car_01
python scripts/uart_monitor_view.py --record results/car_01 --output car_01.html
python scripts/uart_monitor_cli.py retry-save --output results/car_01
```

完成条件：原始字节与时间索引配对，配置及固件摘要已记录；正常用例采集和协议状态为 PASS，注入用例产生对应事件，窗口回读通过。保存失败时运行最后一条命令重试。结束时停止采集、确认缓存排空、保存结果，通过 Linux 正常关机后再断电；重新上电按首次加载步骤恢复并新建会话。

15 总结与展望

技术成果

本系统完成只读 UART 接入、硬件时间戳、配置化解析、八路规则判断、连续事件归档与可视化回放。真实小车、二进制测试流和 LAB 文本分别验证设备接入、同帧多规则处理及协议复用；过载与保存重试实现采集质量和文件状态的独立管理。

应用范围

当前系统适用于已知 UART 遥测协议的数字状态监测与异常取证，可服务于实验室设备、机器人控制模块及传感器原型的联调。接入参数、硬件字段类型、规则数量及窗口容量由配置和接口契约定义，复杂文本和专用协议通过 PS 解析。平衡车承担实物验证载体的作用，平台的配置、事件与取证接口面向设备复用。

八路判断、持续越限、短毛刺过滤、300 秒小车联合采集、双协议接入及正常关机后的人工恢复均取得实测结果。原始数据、事件和窗口通过统一时间及字节范围关联，可用于复核间歇性异常的发生过程。

扩展方向

后续将以机器人关节和多轴传感器为重点，扩展多通道接收、共同硬件时基和并发触发。以六关节系统为设计示例，每关节记录位置、速度、电流三个 32 位量，按 1 kHz 更新，仅字段数据即为 6×3×4×1000 = 72000 B/s，约为当前小车 2400 B/s 业务负载的 30 倍；该数值用于需求估算。随着惯性、力觉等数据加入，平台需要同时处理多源输入、时序关系和持续取证。

扩展路径首先是增加独立采集通路、统一时间索引与链路延迟标定，随后采用批量传输、DMA 及分级缓存支撑持续记录，并按新负载实测吞吐、丢失计数与触发时延。FPGA 承担接收、时间标记和并行规则，PS 完成可配置的分析与归档。在关节过流、姿态跳变或多轴反馈不同步等调试任务中，事件窗口可帮助工程师观察异常传播和恢复过程，减少依赖现场反复重现。

这一方向将现有单通道验证闭环推进到多源精密设备调试：以共享时基保持观测关系，以并发硬件保持持续处理，以可配置协议和统一记录降低新设备接入的重复开发工作，为实验室与中小研发团队提供可逐步扩展的工程工具。

参考资料

[1] AMD. Zynq 7000 SoC Technical Reference Manual UG585. https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM

[2] PYNQ Project. Overlay Tutorial. PYNQ 2.1 Documentation. https://pynq.readthedocs.io/en/v2.1/overlay_design_methodology/overlay_tutorial.html

[3] 本项目技术资料. URX2 可配置接收、寄存器接口与并行规则设计说明.

[4] 本项目技术资料. CAR-TLM1 遥测与 LAB 文本协议定义及配置示例.

[5] 本项目技术资料. 连续事件取证、结果归档与保存重试设计说明.

[6] 本项目技术资料. Notebook 实时监测与离线回放界面设计说明.

[7] ABB. The right moves. ABB Review, 2023. https://new.abb.com/news/detail/102761/the-right-moves

[8] Universal Robots. UR Log Viewer Manual. https://www.universal-robots.com/articles/ur/robot-care-maintenance/ur-log-viewer-manual/

[9] Beckhoff. TwinCAT 3 NC PTP Scope integration. https://infosys.beckhoff.com/content/1033/tf50x0_tc3_nc_ptp/10541536523.html

[10] Bédard C, Lütkebohle I, Dagenais M. ros2_tracing: Multipurpose Low-Overhead Framework for Real-Time Tracing of ROS 2. IEEE Robotics and Automation Letters, 2022, 7(3): 6511–6518. https://arxiv.org/abs/2201.00393

[11] NI. Optimizing your LabVIEW FPGA VIs Parallel Execution and Pipelining. https://www.ni.com/en/shop/electronic-test-instrumentation/add-ons-for-electronic-test-and-instrumentation/what-is-labview-fpga-module/optimizing-your-labview-fpga-vis--parallel-execution-and-pipelin.html

[12] NI. CompactRIO Waveform Reference Library. https://www.ni.com/en/support/documentation/supplemental/09/ni-compactrio-waveform-reference-library.html